CFG to PDA example:

GNF : 1st symbol terminal (single terminal only) & after that any no of non-terminal symbols. [one length : terminal, more than one length : terminal, Non-terminal]

① $S \to aSa \mid asb \mid a$

as there is no null productions & no unit productions, we will check whether given CFG is in GNF form or not.

- given CFG is not in GNF

$S \to aSa$ } → Not in GNF bcz 1st symbol is terminal, after that non terminal, but 3rd symbol is again terminal symbol. so, both are not in GNF.
$S \to asb$ }

so, obtain GNF

$S \to aSA \mid aSB \mid a$

$A \to a$

$B \to b$

$G = (V, T, P, S)$.

$V = \{S, A, B\}$

$T = \{a, b\}$

$P = \{S \to aSA \mid aSB \mid a$

$A \to a$

$B \to b\}$

ch-4 (8)

Transition of PDA

(1) For all CFG productions

i) $S \to aSA$

$\delta(q, \epsilon, S) = (q, aSA)$

ii) $S \to aSB$

$\delta(q, \epsilon, S) = (q, aSB)$

iii) $S \to a$ $\delta(q, \epsilon, S) = (q, a)$

iv) $A \to a$

$\delta(q, \epsilon, A) = (q, a)$

v) $B \to b$

$\delta(q, \epsilon, B) = (q, b)$

(2) For all terminals

(i) for 'a'

$\delta(q, a, q) = (q, \epsilon)$

(ii) for 'b'

$\delta(q, b, b) = (q, \epsilon)$

* TF of PDA

1) $\delta(q, \epsilon, S) = (q, aSA)$

2) $\delta(q, \epsilon, S) = (q, aSB)$

3) $\delta(q, \epsilon, S) = (q, a)$

4) $\delta(q, \epsilon, A) = (q, a)$

5) $\delta(q, \epsilon, B) = (q, b)$

6) $\delta(q, a, a) = (q, \epsilon)$

7) $\delta(q, b, b) = (q, \epsilon)$

(2) CFG → $G = (\{S, A\}, \{0, 1\}, P, S)$

P is defined by:

$S \to 0S \mid A$

$A \to 1A0 \mid S \mid \epsilon$

CFG to PDA

CH-9 (10)

1) find nullable variables

A, S

Remove null productions

$S \to 0S \mid 0 \mid A$

$A \to 1A0 \mid 10 \mid S$

2) find unit productions

A, S on right side

- remove unit productions.

① $S \to 0S \mid 0 \mid 1A0 \mid 10 \mid S$

$A \to 1A0 \mid 10 \mid S$

② $S \to 0S \mid 1A0 \mid 10 \mid 0$

$A \to 1A0 \mid S \mid 10$

③ $S \to 0S \mid 1A0 \mid 10 \mid 0$

$A \to 1A0 \mid 0S \mid 1A0 \mid 10 \mid 0 \mid 10$

3) obtain GNF

$S \to 0S \mid 1AB \mid 1B \mid 0$

$B \to 0$

$A \to 1AB \mid 0S \mid 1AB \mid 1B \mid 0 \mid 1B$

4) Transition of PDA

i) transition for all CFG productions

① $\delta(q, \epsilon, S) = (q, 0S)$

② $\delta(q, \epsilon, S) = (q, 1AB)$

③ $\delta(q, \epsilon, S) = (q, 1B)$

④ $\delta(q, \epsilon, S) = (q, 0)$

⑤ $\delta(q, \epsilon, B) = (q, 0)$

⑥ $\delta(q, \epsilon, A) = (q, 1AB)$

⑦ $\delta(q, \epsilon, A) = (q, 0S)$

⑧ $\delta(q, \epsilon, A) = (q, 1AB)$

⑨ $\delta(q, \epsilon, A) = (q, 1B)$

⑩ $\delta(q, \epsilon, A) = (q, 0)$

⑪ $\delta(q, \epsilon, A) = (q, 1B)$

ii) transition for all terminals

① $\delta(q, 0, 0) = (q, \epsilon)$

② $\delta(q, 1, 1) = (q, \epsilon)$

PDA → CFG

Q. Obtain CFG

$M = (\{q_0, q_1\}, \{0,1\}, \{z_0, X\}, \delta, z_0, q_0, \emptyset)$

(Q, Σ, Γ, δ, z_0, q_0, F)

z_0 → initial stack top

δ is defined as

1) $\delta(q_0, 1, z_0) = (q_0, Xz_0)$ } push
2) $\delta(q_0, 1, X) = (q_0, XX)$ } push
3) $\delta(q_0, 0, X) = (q_1, X)$ → read
4) $\delta(q_1, \epsilon, z_0) = (q_0, \epsilon)$ → pop
5) $\delta(q_1, 1, X) = (q_1, \epsilon)$ → pop
6) $\delta(q_1, 0, z_0) = (q_0, z_0)$ → read

⟹ PDA → CFG

↓

$G = (V, T, P, S)$

Q = {q_0, q_1} → states

Γ = {z_0, X} → stack alphabets / elements

1) Start symbol (S)

$S \rightarrow [q_0, z_0, q_0]$

$S \rightarrow [q_0, z_0, q_1]$

2) Terminals (T)

T = {0, 1}

3) Non Terminals (V)

$[q_0, z_0, q_0] = A$	$[q_0, X, q_0] = E$
$[q_0, z_0, q_1] = B$	$[q_0, X, q_1] = F$
$[q_1, z_0, q_0] = C$	$[q_1, X, q_0] = G$
$[q_1, z_0, q_1] = D$	$[q_1, X, q_1] = H$

4) Production rules (P)

1) Read : when top stack at both sides is same.
2) Push : when top stack at both sides is not same
3) Pop : when top stack at both sides is not same and ε at right side.

1) For Push → 4 productions.

(i) $\delta(q_0, 1, Z_0) = (q_0, XZ_0)$

$[q_0, Z_0, q_0] \rightarrow 1\,[q_0, X, q_0]\,[q_0, Z_0, q_0]$

$[q_0, Z_0, q_0] \rightarrow 1\,[q_0, X, q_1]\,[q_1, Z_0, q_0]$

$[q_0, Z_0, q_1] \rightarrow 1\,[q_0, X, q_0]\,[q_0, Z_0, q_1]$

$[q_0, Z_0, q_1] \rightarrow 1\,[q_0, X, q_1]\,[q_1, Z_0, q_1]$

Production rule for Push

$S \rightarrow A \mid B$ → why? → because in first step, it is given

$S \rightarrow [q_0, Z_0, q_0]$

$S \rightarrow [q_0, Z_0, q_1]$

$A \rightarrow 1EA \mid 1FC$

$B \rightarrow 1EB \mid 1FD$

(ii) $\delta(q_0, 1, X) = (q_0, XX)$

$[q_0, X, q_0] \rightarrow 1\,[q_0, X, q_0]\,[q_0, X, q_0]$

$[q_0, X, q_0] \rightarrow 1\,[q_0, X, q_1]\,[q_1, X, q_0]$

$[q_0, X, q_1] \rightarrow 1\,[q_0, X, q_0]\,[q_0, X, q_1]$

$[q_0, X, q_1] \rightarrow 1\,[q_0, X, q_1]\,[q_1, X, q_1]$

$E \rightarrow 1EE \mid 1FG$

$F \rightarrow 1EF \mid 1FH$

2) For Read → 2 productions.

i) $\delta(q_0, 0, X) = (q_1, XX)$

$[q_0, X, q_0] \rightarrow 0\,[q_1, X, q_0]$

$[q_0, X, q_1] \rightarrow 0\,[q_1, X, q_1]$

$E \rightarrow 0G$

$F \rightarrow 0H$

ii) $\delta(q_1, 0, Z_0) = (q_0, Z_0)$

$[q_1, Z_0, q_0] \rightarrow 0\,[q_0, Z_0, q_0]$

$[q_1, Z_0, q_1] \rightarrow 0\,[q_0, Z_0, q_1]$

$C \rightarrow 0A$

$D \rightarrow 0B$

3) For Pop → 1 production

i) $\delta(q_1, 1, X) = (q_1, \epsilon)$

$[q_1, X, q_1] \rightarrow 1$

$H \rightarrow 1$

$C \rightarrow \epsilon$

ii) $\delta(q_1, \epsilon, Z_0) = (q_0, \epsilon)$

$[q_1, Z_0, q_0] = \epsilon$

$S \rightarrow A \mid B$

$A \rightarrow 1EA \mid 1FC$

$B \rightarrow 1EB \mid 1FD$

$E \rightarrow 1EE \mid$ ~~$1FG$~~

$F \rightarrow 1EF \mid 1FH$

~~$E \rightarrow 0G$~~

$F \rightarrow 0H$

$C \rightarrow 0A$.

$D \rightarrow 0B$

$H \rightarrow 1$

$C \rightarrow \epsilon$.

⇒ check useful symbols:

G is not useful if the terminal generates non terminal symbols, then it can be called as useful symbol.

Sudhakar Atchala

Pumping lemma for CFL

↳ it is used to prove a language as not CF.

- Let 'L' be context free langu. Let "n" be an integer constant.
- select a string 'z' from 'L' such that $|z| \geq n$
- Divide the string z into 5 parts, pqrst such that $|qrs| \leq n$, $|qs| \geq 1$

for $i \geq 0$, $pq^i r s^i t$ is in 'L'

Q. show that $L = \{a^n b^n c^n \mid n \geq 1\}$ is not a CFL.

Sol^n let 'L' is a CFL

$L = \{abc, aabbcc, aaabbbccc, aaaabbbbcccc, \ldots\}$

- Let string 'z' from L is : aaabbbccc, so, n = 3

(i) now, check : $|z| \geq n$

$|aaabbbccc| \geq 3$

$9 \geq 3$ ✓

(ii) Divide z into 5 parts

aa a bb bccc

p q r s t

p = aa r = b t = bccc

q = a s = b

(iii) check $|qrs| \leq n$

$|abb| \leq 3$

$3 \leq 3$ ✓

(iv) check $|qs| \geq 1$

$|ab| \geq 1$

$2 \geq 1$ ✓

(v) for $i \geq 0$, $pq^i r s^i t$

$i = 0$, $pq^0 r s^0 t$

$= prt$

$= aabbccc \notin L$ ∴ L is not CFL

Note: we have to check until we get string which does not belong to the language 'L'

Q. Prove that $L = \{a^p \mid p \text{ is a prime number}\}$ is not CFL

Sol^n: Let L be a CFL

$\therefore L = \{aa, aaa, aaaaa, aaaaaaa, \ldots\}$

$z = aaaaa$, for $p = 5$

(i) $|z| \geq p \rightarrow 5 \geq 5$ ✓

(ii) $\underset{p}{a}\ \underset{q}{a}\ \underset{r}{a}\ \underset{s}{a}\ \underset{t}{a}$

(iii) $|qrs| \leq p$

$|aaa| \leq 5$

$3 \leq 5$ ✓

(iv) $|qs| \leq p$

$|aa| \leq 5$

$2 \leq 5$ ✓

(v) $pq^i r s^i t$, $i \geq 0$

$i = 0 \Rightarrow aa^0 aa^0 a$

$= aaa \in L$

$i = 1 \Rightarrow aa^1 aa^1 a \in L$

$i = 2 \Rightarrow aaaaaaa \in L$

$i = 3 \Rightarrow pq^3 r s^3 t$

$aa^3 r a^3 t$

$aaaaaaaaa \notin L$

bcz 9 is not a prime number

$\therefore$ given language L is not CFL

How to know that given language is CFL or not?

- balanced parentheses
- equal no of a's & b's → $a^n b^n c^m$ is CFL, $a^n b^n$ is CFL
- not equal no of a's, b's & c's
  - ↳ i.e. $a^n b^n c^n$ is not CFL

- if CFG can be constructed from that language, then CFL
- if lang. can be accepted by PDA, then it is CFL.

Intersections of CFL

→ Intersection of two languages L_1 and L_2 is :

$L_1 \cap L_2 = \{\omega \mid \omega \in L_1 \text{ and } \omega \in L_2\}$

- CFLs are NOT closed under intersection.

ex $L_1 = \{a^n b^n c^m \mid n, m \geq 0\}$

$L_2 = \{a^m b^n c^n \mid m, n \geq 0\}$

→ $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$

↳ How? from L_1 : no of a's = no of b's, no of c's is free → $a = b$

from L_2 : no of b's = no of c's, no of a's is free → $b = c$

→ combine both : $a = b$, $b = c$ → $\therefore a = b = c$

$\therefore L_1 \cap L_2 = \{a^m b^n c^n \mid n \geq 0\}$

here, $L_1 \cap L_2$ is not a CFL → it can be proved using pumping lemma.

Since, intersection of two CFLs results in a non-CFL, CFLs are not closed under intersection.

Complements of CFL

what is complement ?

for any language L: $L^c = \{w \mid w \notin L\}$

means - all strings NOT in L

→ using contradiction :

Step 1 : Assume : CFLs are closed under complement.

ex $L = \{a^n b^n \mid n \geq 0\}$ → this is CFL

Step 2 : Complement of $L = a^n b^n$ is → all strings NOT of the form $a^n b^n$

So, L^c contains : aab, abb, ba, abab
↓ a≠b ↓ wrong order

→ as there are too many conditions for L^c to check : unequal counts, wrong order, mixed structure, ...,

we can say that CFL is not closed under complement.

i.e. If L_1 is a CFL then L_1' may or may not be CFL.

17(2)

* Union of CFLs

$L_1 = \{a^n b^n\}$ → $L_1 = \{ab, aabb, \ldots\}$

$L_2 = \{a^n b^{2n}\}$ $L_2 = \{abb, aabbbb, \ldots\}$

→ $L_1 \cup L_2$: A string is accepted if it belongs to: either L_1 or L_2

$L_1 \cup L_2 = \{a^n b^n\} \cup \{a^n b^{2n}\}$

$= \{ab, aabb, aaabbb, \ldots\} \cup$
$\{abb, aabbbb, aaabbbbbb, \ldots\}$

$= \{ab, aabb, aaabbb, abb, aabbbb, \ldots\}$

Why this is CFL ?

When a string comes, we have two choices:

ex aabb
- count a's
- match equal b's

} accepted by $L_1 \cup L_2$

ex aabbbb
- count a's
- match double b's

} accepted by $L_1 \cup L_2$

∴ CFL is closed under union.

* Kleene closure of CFL → L^*

$L = \{a^n b^n\} = \{ab, aabb, aaabbb, \ldots\}$

L^* = repeat strings from L

ex abab, abaabb, ababab, ...

→ Take string : abaabb

break it : ab | aabb

first block : ab
- push a
- push b

stack becomes empty

second block : aabb
- push a a
- pop b b

stack becomes empty again

→ After finishing one block, stack becomes empty, so machine is ready to process next block.

∴ CFL is closed under Kleene closure star.

* Concatenation of CFLs

$L_1 = \{a^n b^n\} = \{ab, aabb, \ldots\}$

$L_2 = \{c^m d^m\} = \{cd, ccdd, \ldots\}$

$L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$

→ $L_1 = ab$, $L_2 = cd$ } → $L_1 L_2 = abcd$

$L_1 = ab$, $L_2 = ccdd$ } → $L_1 L_2 = abccdd$

→ $L_1 L_2$ is CFL. why ?

step 1 : read first part from L_1 : match a's with b's

step 2 : then read second part from L_2 : match c's with d's

→ we are doing : first solve one problem (1st condition finished), then solve another (2nd condition starts)

→ ∴ CFL is closed under concatenation.

→ Grammar Idea :

$S \rightarrow S_1 S_2$

S_1 generates $a^n b^n$

S_2 generates $c^m d^m$

Non-CFL

→ Non-Context Free languages are languages that cannot be generated by any CFG and cannot be accepted by PDA.

CFL → can be handled using one stack

Non-CFL → needs more than one stack / more memory

So, if a language has too many dependencies - it is Non-CFL

ex $L = \{a^n b^n c^n \mid n \geq 0\}$

as $a = b = c$, that's two dependencies: match 'a' with b
match 'b' with c
∴ one stack can't handle this.

ex $L = \{\omega\omega \mid \omega \in \{a,b\}^*\}$

why NOT CFL?

- needs to remember entire first half;
Then match exactly with second half.

* How to identify Non-CFL?

A language is Non-CFL if:

- it requires multiple equalities
- it fails Pumping lemma for CFL
- it can't be handled by single stack.